

CANDY DIALOGUE ENGINE

Update

1.1.0

This document explains how to update Candy Dialogue Engine in a Godot project.

To install Candy Dialogue Engine inside a new project, see **Installation_Instructions.pdf** instead.

If you need help with installation or upgrading, please feel welcome to post on our GitHub discussion board, or in the comments of any of our Youtube videos!

You can also contact us through our Contact page on our website if you prefer.

We are always happy to help!

Updating from 1.0 to 1.1:

Candy Dialogue Engine 1.1 has made some changes to its files and folders.

If your project has a 1.0 version of Candy Dialogue Engine, and you are now upgrading to 1.1, you must first follow the instructions in this section.

Dialogues also use a new format, and will need to be upgraded from 1.0 to 1.1 as well: we made a converter to automate the process, and instructions are provided further down.

If you are upgrading from a 1.1 version to another 1.1 version, skip these instructions and go to "Updating" at the end of this document.

We provide a patcher tool to automatically update the default Candy Dialogue Engine files and folders. Note that the patcher only updates files or folders if they are found in the default locations.

If your project uses any custom folders or paths instead of the default Candy_DE folders, here is what you need to know:

It's safe to run the patcher. It will only modify what matches the Candy Dialogue Engine defaults. It won't break anything.

You'll need to modify some things manually. Follow the Automatic Patching instructions below first, then follow the Manual Patching instructions.

Automatic Patching:

1. **Make a backup of your entire project before you continue!**
2. Download the latest version of Candy Dialogue Engine (1.1.x).
3. In the downloaded files, in the same folder as the Candy_DE and the Documents folders, you will see a file named '1_0_to_1_1_patcher.gd'. This is the patcher.
4. Move the patcher file to your Godot project folder. Place it in the root folder, where the file 'project.godot' lives. **DO NOT** place it in Candy_DE or in the '.addons' folder: if the patcher is not in the project's root folder, it will fail.
5. Open your project in the Godot Editor.
6. In the FileSystem panel, look for the patcher - '1_0_to_1_1_patcher.gd'.
7. Right-click the patcher, and select 'Run'.
8. Wait for the patcher to finish. Various messages will print in the Output console. If you see errors or warnings about missing files or folders, it likely means you didn't have those files and folders, or you renamed or moved them. It's not necessarily a problem.
9. When the patcher finishes its job, it will print "Migration complete!" in the Output console. You can then proceed to the **Updating** section further down. You can also delete the patcher from your project.

If you saw error messages about missing files or folders, it can mean several things. Don't worry, it's probably fine!

- You may have deleted some folders because you weren't using them (or you never created them): all good, you don't need to do anything else!
- You may have moved or renamed files or folders, or you use different ones: see **Manual Patching** below, in that case.

The patcher worked perfectly during our testing, but it doesn't hurt to check the results on your side. Here is what it should have done:

1. All folders in Candy_DE/Scenes should have been renamed to replace spaces with underscores. E.g. 'Background Scenes' → 'Background_Scenes'
2. 'Testing Scene.tscn' should have been renamed to 'Testing_Scene.tscn' (underscore).
3. Inside Candy_DE/Media:
 - All subfolders named "Sprite Frames", "Animation Libraries" and "Button Textures" should now have the space replaced by an underscore (e.g. "Sprite_Frames").
 - These folders can be found everywhere: inside all the actor folders inside 'Busts', 'Portraits' and 'Sprites', inside all the layer folders in 'Backgrounds', and possible in subfolders in 'Images'.

- You don't need to check all the folders (there can be a lot). But if you get an error during a dialogue later, where the code can't find a path with "Sprite_Frames", "Animation_Libraries" or "Button_Textures", it could mean one of those folders was not renamed correctly by the patcher.
4. Inside **Candy_DE/Scripts**:
 - The folders 'Actor Nodes', 'Media Players' and 'UI Elements' should have been renamed to use an underscore instead of space.
 - The 'User_Data' folder should have been created.
 - **Candy_Database.gd**, **Candy_Localizations.gd**, **Candy_Functions.gd** and **Candy_Properties.gd** should have been moved to the 'User_Data' folder (their matching .uid files should also have been moved there).
 5. Open **Candy_Database.gd**, **Candy_Properties.gd**, and **Candy_Functions.gd**:
 - At the top of each script, it says 'extends' followed by a path.
 - Check that the path contains 'User_Data' after 'Candy_DE/Scripts/'
 - **Candy_Localizations.gd** extends 'Node': this is normal.
 6. In **Candy_Database.gd**, in the '**actors**' dictionary:
 - For each actor, the key "**Bubble Exempt**" should have been replaced by "**Bubbles**".
 - If the value of "**Bubble Exempt**" was true, the value of "**Bubbles**" should now be -1. If the value was false, it should now be 0.
 - If some of your actors don't have the "**Bubbles**" key, it means they did not have the "**Bubble Exempt**" key before.
 - "**Bubbles**" is not mandatory. Nothing breaks if actors don't have it. The engine treats the missing "**Bubbles**" key as if it had a value of 0.
 7. In **Candy_Properties.gd**, the following paths should have been updated to replace spaces with underscores:
 - `var bust_scenes_folder = "res://Candy_DE/Scenes/VN_Scenes"`
 - `var bg_scenes_folder = "res://Candy_DE/Scenes/Background_Scenes"`
 - `var input_menus_folder = "res://Candy_DE/Scenes/Input_Menus"`
 - `var choice_menus_folder = "res://Candy_DE/Scenes/Choice_Menus"`
 - `var choice_categories_folder = "res://Candy_DE/Scenes/Choice_Categories"`
 - `var choice_buttons_folder = "res://Candy_DE/Scenes/Choice_Buttons"`
 - **Note:** the paths should NOT have updated if you customized them and they didn't match the original default paths.
 8. Finally, the patcher should also have updated the candy_de autoload in Project Settings to match the new path, but only if **Candy_Functions.gd** was originally in **Candy_DE/Scripts**. If you moved it elsewhere, the autoload won't be updated by the patcher.

Notice: if you added references to any of the paths or file names that are changed in 1.1 in your own code, you will need to update those references.

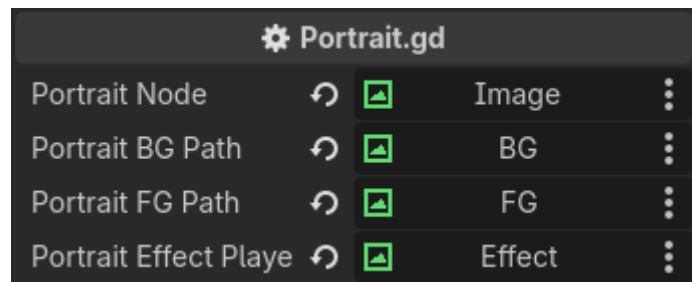
Manual Patching:

In all cases, do the following:

#1: In **Candy_Database.gd**, add the '**current_player**' variable:

```
#^ Player data:  
#? Which entry in the actors dictionary represents the player's data.  
#? Used by variants based on player properties (e.g. player gender).  
var current_player = "Player"
```

#2: Open all your UI scenes, select the root nodes, and assign nodes manually to the exported variables that require nodes (for the Bark scenes, select "BarkBubble" instead of the root node).



If you are unsure which nodes to assign, look at the Candy_Default.tscn scenes for guidance: they are updated with the nodes assigned.

This is not necessary for scenes that use custom scripts that are not updated by Candy_DE v1.1.

If you use custom folders or paths instead of the default Candy_DE folder structure, look at the checklist above. You MUST make the following changes manually:

#1: Rename all "Sprite Frames", "Animation Libraries" and "Button Textures" folders to "Sprite_Frames", "Animation_Libraries" and "Button_Textures".

#2: In **Candy_Database.gd**, in the '**actors**' dictionary, use your code editor's Find & Replace tool to replace the following (include the quotation marks):

"Bubble Exempt": true → "Bubbles": -1

"Bubble Exempt": false → "Bubbles": 0

That's it. The other folders or files don't need to be renamed if you use a custom folder structure, and the paths in the scripts don't need to be changed (the patcher will have changed the default paths in the scripts if you still used any).

Note that while Candy Dialogue Engine supports a custom folder structure / custom paths, it makes updates require a bit more work on the user's end (as you probably noticed). It also makes it

difficult for us to provide support or anticipate special cases. For this reason, it's recommended to use the default folder structure, but Candy Dialogue Engine will continue allowing custom paths.

Optional Changes:

Version 1.1 no longer requires the "Dialogue_Speed", "Dialogue_Slow" and "Dialogue_Skip" Input Map actions (in Project Settings → Input Map).

These actions still work, they were not removed. But if you don't use them in your project, you can now remove them from the Input Map if you want to.

Dialogues Files:

Exported dialogue files made in version 1.0 will need to be converted to version 1.1 in order to work.

This can be done easily with the Converter tool integrated into Candy Dialogue Creator 1.1.

See the Candy Dialogue Creator documentation for instructions.

Updating

If you are updating from 1.0 to 1.1, please first follow the instructions in "Updating from 1.0 to 1.1" above. Then follow the instructions here.

Updating isn't difficult, but it carries the risk of losing custom data that you added to the Candy Dialogue Engine files if you don't pay attention.

To avoid problems, it's important that you follow the instructions below carefully.

You should also read update notes carefully, in case an update has special instructions.

1. **Make a backup copy of your project's Candy_DE folder!** This guarantees you can't lose any data if you make a mistake. A backup of your entire project is a good idea, just to be safe.
2. Download the latest version of Candy Dialogue Engine.
3. **DO NOT delete your project's Candy_DE folder!**
4. Copy the downloaded **Candy_DE** folder to your project's **Candy_DE** folder. Overwrite files if prompted.
5. **When updating, DO NOT copy the files inside 'First_Install' to 'Candy_DE'.**

Notes:

After updating, if you see any UID errors in the Output console in Godot, upgrade the project files:

"Project" at the top of the Godot editor → "Tools" → "Upgrade project files..."

If we ever modify Candy_UI.tscn, Testing_Scene.tscn, or the scripts in User_Data, we will mention this in the update notes, and we'll give you precise instructions on what to do. These files are not normally modified between updates.

Special Note:

In 1.0, the Candy_DE/Scenes subfolders came with scenes named 'Default.tscn'.

In 1.1, these subfolders now come with scenes named 'Candy_Default.tscn'.

After upgrading from 1.0 to 1.1 you will have both. This is good:

- **Scenes that begin with 'Candy_'** are just templates. Don't modify them or use them in your game: copy them, and use/modify the copies.
- **Scenes that DON'T begin with 'Candy_'** are the scenes you actually use in your game (either copied from the templates, or created entirely by yourself).

This way, when you update Candy Dialogue Engine in the future, the Candy_ scenes can be safely overwritten.

But why not just treat "Default.tscn" as a template, and use other names for the scenes your game uses?

Because sometimes, dialogue commands require that Candy DE load UI scenes. If you didn't specify which scene to load or use, it will fallback to loading the scene named "Default.tscn" (if a scene by that name exists).

This meant that if template scenes were named "Default.tscn", you couldn't use that name as a fallback scene... And if you renamed the template scene to something else to free up the "Default.tscn" name for a custom scene, the next Candy DE update would overwrite that custom scene... This change will therefore resolve this conflict.

Additionally, if we ever provide multiple template scenes for a given UI element, we'll also prefix these templates with "Candy_" (e.g. "Candy_Blue_Theme.tscn"). This guarantees that as long as you don't name your own scenes "Candy_", our scenes will never conflict and overwrite them.

We hope that's clear... If not, always feel free to ask questions on our [forum](#).

Summary:

Updates usually only require that you copy the new Candy_DE folder to your project, and overwrite files.

Never modify the 'Candy_' scenes directly, or use them in your game: make copies, and use/modify those. 'Candy_' scenes are overwritten between updates.

The **Candy_Engine.gd** script is also overwritten between updates. This is where we make most changes to Candy Dialogue Engine. **If you modify Candy_Engine.gd, always keep a copy of your changes, then re-add them manually to the new, updated file.**

If an update involves anything else you need to know, it will be clearly documented, with precise instructions.